FlexQL Database Engine

Detailed Design and Architecture Document

April 7, 2026

Name: **Nilesh Gupta**
Roll No.: 25CS60R64
GitHub Repository: github.com/Nilesh0711/FLEXQL

Abstract

FlexQL is a compact relational database engine implemented primarily in C with benchmark and test drivers in C++. The current system exposes a loopback-only client/server interface, persists table data on disk, maintains a per-table primary-key B+ tree index, replays a write-ahead log during startup, and uses an in-memory LRU cache for selected query results. This document describes the design as implemented in the repository, including the parser, executor, storage format, concurrency model, recovery path, query optimization choices, and operational limitations.

Contents

1 System Overview **4**

2 Goals and Non-Goals **4**
2.1 Goals 4
2.2 Non-Goals 4

3 Supported SQL Surface **5**

4 High-Level Architecture **5**
4.1 Module Map 5
4.2 Runtime Components 6

5 Request Lifecycle **7**
5.1 End-to-End Flow 7
5.2 Client/Server Protocol 8

6 Parser Design **9**
6.1 Tokenizer 9
6.2 Statement Structures 9
6.3 Condition Representation 10

7 Execution Engine **10**
7.1 Dispatcher 10

7.2 SELECT Execution Paths . . . 10
7.2.1 Path 1: Cache Hit . . . 10
7.2.2 Path 2: Primary-Key Equality Lookup . . . 10
7.2.3 Path 3: Sequential Scan . . . 11
7.2.4 Path 4: Indexed Join . . . 11
7.2.5 Path 5: Nested-Loop Join . . . 11
7.3 Projection Handling . . . 11
7.4 Ordering . . . 12

8 Storage Engine **12**
8.1 Directory Layout . . . 12
8.2 Metadata File . . . 12
8.3 Row File Format . . . 12
8.4 Insert Path . . . 13
8.5 Scan Path . . . 13

9 Type System and Validation **13**

10 Primary-Key Index Design **14**
10.1 Index Role . . . 14
10.2 Index File Structure . . . 14
10.3 Lookup and Insert Behavior . . . 14
10.4 Index Diagram . . . 14

11 Caching Strategy **15**
11.1 Purpose . . . 15
11.2 Structure . . . 15
11.3 Policy . . . 16
11.4 Tradeoff . . . 16

12 Expiration Semantics **16**

13 Write-Ahead Logging and Recovery **16**
13.1 WAL Format . . . 16
13.2 When WAL Is Used . . . 16
13.3 Recovery Process . . . 17
13.4 Recovery Diagram . . . 17

14 Concurrency Model **18**
14.1 Server Concurrency . . . 18
14.2 Locking Policy . . . 19
14.3 Cache Synchronization . . . 19
14.4 Concurrency Diagram . . . 19

15 Server and Client API **20**
15.1 Public API . . . 20
15.2 Behavior . . . 20

16 Testing and Benchmarking **21**
16.1 Regression Test Driver . . . 21

16.2 Benchmarks 21

17 Known Limitations **21**

18 Summary **22**

1 System Overview

FlexQL is designed as a focused SQL execution engine rather than a full SQL database. The repository implements a narrow but coherent stack:

- a public C API used by the REPL, tests, and benchmarks,
- a multithreaded TCP server bound to loopback,
- a handwritten SQL tokenizer and parser,
- an executor that chooses between index lookup, sequential scan, indexed join, and nested-loop join,
- a storage layer that persists schema metadata, row files, and primary-key index files,
- a small in-memory LRU cache for stable `SELECT` results,
- a simple write-ahead log used for crash recovery.

The system prioritizes:

- simple persistence,
- predictable execution paths,
- moderate concurrency for concurrent readers,
- a clear, teachable implementation structure.

2 Goals and Non-Goals

2.1 Goals

- Support a practical subset of SQL sufficient for table creation, insertion, selection, filtering, ordering, and joins.
- Persist all data on disk across process restarts.
- Speed up equality lookup on the first column of each table using a persisted B+ tree.
- Allow multiple client connections through a single server process.
- Recover from interrupted mutating statements using WAL replay on startup.
- Keep the codebase modular enough for coursework, experimentation, and benchmarking.

2.2 Non-Goals

- Full SQL compliance.
- Distributed operation or remote host support.
- Transactions with commit/rollback semantics.
- Predicate-based row deletion or row updates.
- Secondary indexes, query planning statistics, or cost-based optimization.
- MVCC or fine-grained lock management.

3 Supported SQL Surface

The parser and executor currently recognize the following statement families:

Statement	Behavior in Current Code
`CREATE TABLE [IF NOT EXISTS] ...`	Creates metadata, row file, and primary-key index file.
`INSERT INTO ... VALUES (...), (...)`	Supports single-row and multi-row insertion. Values are validated against declared types before persistence.
`SELECT ... FROM ...`	Supports projection, `*`, optional `WHERE`, optional `INNER JOIN`, and optional `ORDER BY`.
`DELETE FROM table`	Clears all rows in the target table by truncating the row file and resetting the index.
`RESET table`	Alias for the same table-clear behavior as `DELETE FROM table`.

Supported predicate operators are:

- `=`
- `!=`
- `<`
- `<=`
- `>`
- `>=`

Important behavioral rules:

- The first column of every table is treated as the primary indexed lookup key.
- `ORDER BY` is valid only if the ordered column appears in the `SELECT` output list.
- If a table has a column named `EXPIRES_AT`, rows with expired timestamps are skipped during reads.
- The server accepts only loopback hosts: `127.0.0.1` and `localhost`.

4 High-Level Architecture

4.1 Module Map

Figure 1 is intended to hold the rendered Mermaid architecture diagram for the overall FlexQL system. Export your Mermaid diagram as a PNG or PDF and place it at:

- `diagrams/system_architecture.png`

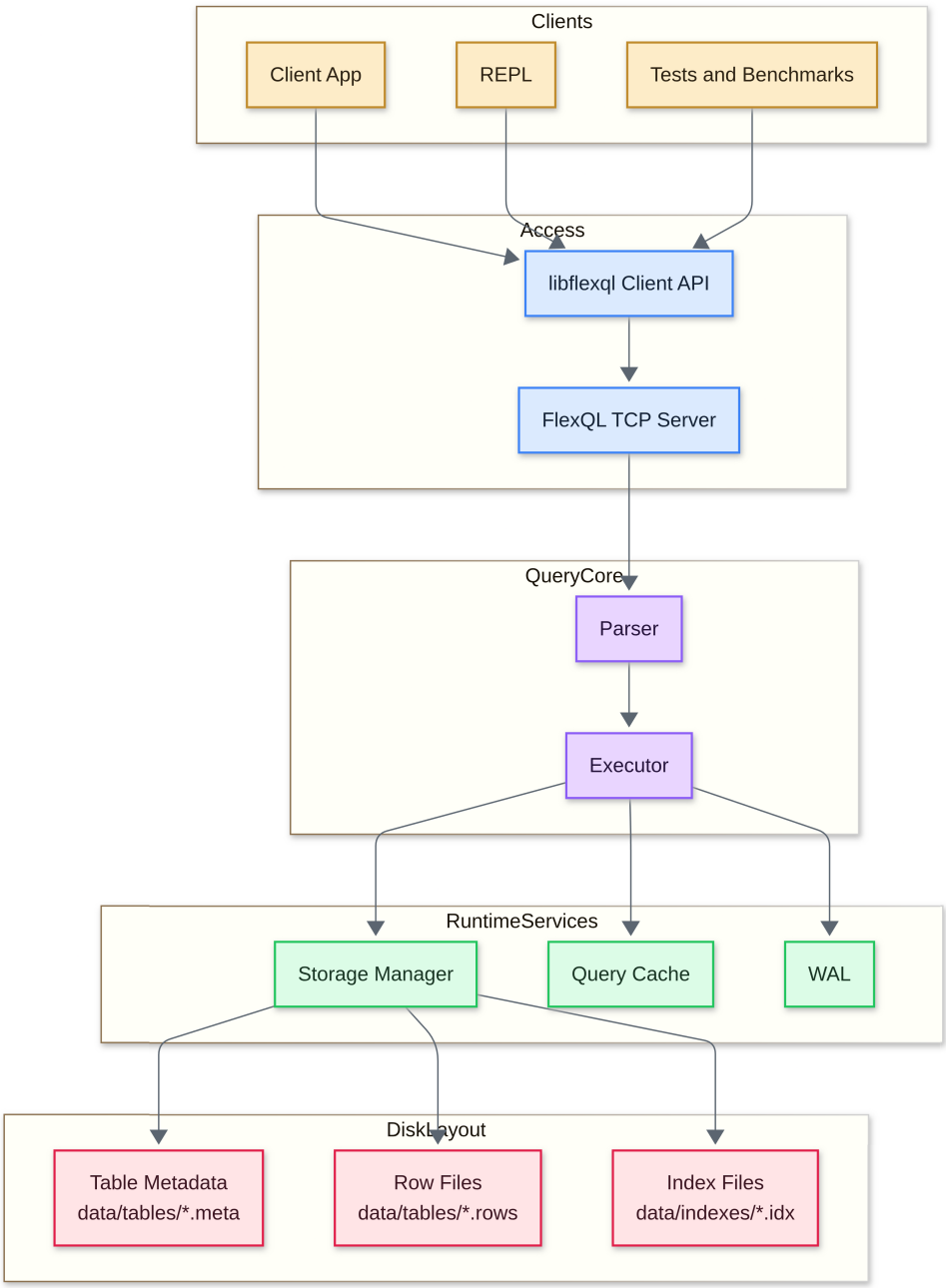

Figure 1: High-level FlexQL architecture showing client access, server runtime, parser, executor, and persistence subsystems.

For convenience, the Mermaid source that matches this section is:

4.2 Runtime Components

- **Client API:** `flexql_open`, `flexql_exec`, `flexql_close`, and `flexql_free`.
- **Server runtime:** initializes storage and cache, replays the WAL, listens on a TCP port, and spawns one thread per accepted client connection.
- **Parser:** tokenizes SQL text and produces a statement structure.
- **Executor:** performs statement-specific work and decides whether to scan, sort, join, or

use the primary index.

- **Storage manager:** owns table metadata in memory and file layout on disk.
- **Index manager:** persists and queries the B+ tree file for the first column of each table.
- **Cache:** stores recent `SELECT` results keyed by SQL text.
- **Expiration helper:** interprets `EXPIRES_AT` as epoch time and suppresses expired rows at read time.

5 Request Lifecycle

5.1 End-to-End Flow

Figure 2 is intended to hold the rendered Mermaid request-flow diagram. Export your Mermaid diagram as a PNG or PDF and place it at:

- `diagrams/request_lifecycle.png`

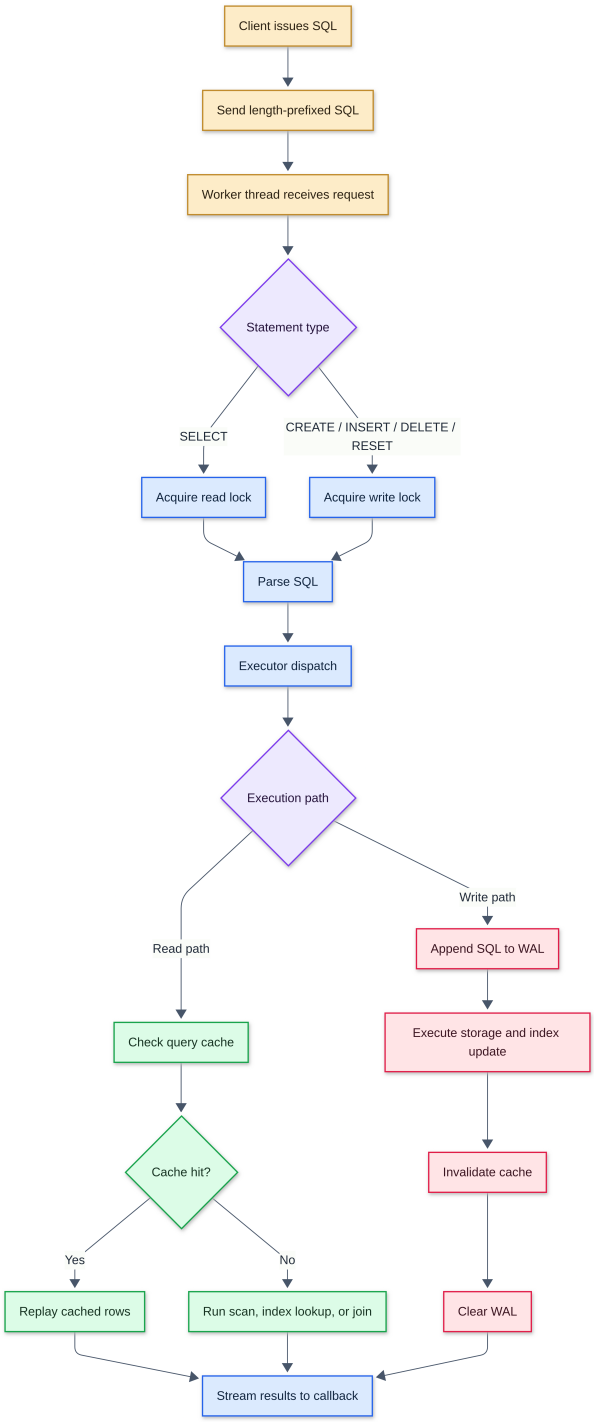

Figure 2: End-to-end request lifecycle from client SQL submission to streamed results.

5.2 Client/Server Protocol

The network protocol is intentionally simple and binary:

- The client sends the SQL statement as `uint32 length + bytes`.
- The server streams results using markers and length-prefixed strings.
- Column names are sent once before row data.

- Each row is serialized column by column.
- Errors are sent as status markers plus an error string.

This protocol avoids text parsing on the wire and allows the server to stream large result sets without first rendering a textual table format.

6 Parser Design

6.1 Tokenizer

The parser uses a handwritten tokenizer in `src/parser/parser.c`. It recognizes:

- identifiers and keywords,
- quoted string literals enclosed in single quotes,
- punctuation such as commas and parentheses,
- comparison operators such as `=`, `!=`, `<=`, `>=`,
- statement terminators such as semicolons.

Each token stores:

- token text,
- whether the token was quoted.

6.2 Statement Structures

The parser maps SQL into a tagged statement structure:

- `FLEXQL_STMT_CREATE`
- `FLEXQL_STMT_INSERT`
- `FLEXQL_STMT_SELECT`
- `FLEXQL_STMT_DELETE`
- `FLEXQL_STMT_RESET`

For `SELECT`, the parser records:

- whether `*` was used,
- projected columns,
- source table,
- join table and join condition if present,
- where condition if present,
- ordered column if present.

6.3 Condition Representation

Conditions are stored as:

- left operand,
- comparison operator,
- right operand,
- whether the right operand was quoted,
- whether the right operand should be treated as a literal instead of a column reference.

This allows the executor to evaluate both:

- literal predicates such as `ID = 2`,
- column-vs-column predicates such as `TEST_USERS.ID = TEST_ORDERS.USER_ID`.

7 Execution Engine

7.1 Dispatcher

Execution begins in `executor_exec_sql`. After parsing, the executor:

- appends mutating statements to the WAL,
- dispatches by statement type,
- invalidates the cache after successful mutations,
- clears the WAL after successful completion.

7.2 SELECT Execution Paths

The most important implementation logic is in `execute_select`. The executor chooses among several paths.

7.2.1 Path 1: Cache Hit

If the query is cacheable and an identical SQL string exists in the LRU cache, cached rows are replayed directly to the callback.

The query is **not** cached when any referenced table contains an `EXPIRES_AT` column, because the visible rows depend on wall-clock time.

7.2.2 Path 2: Primary-Key Equality Lookup

For a single-table query of the form:

```
SELECT ... FROM T WHERE first_column = literal;
```

the executor performs a direct index lookup using `storage_lookup_primary`. This turns a full scan into:

- B+ tree lookup in `table.idx`,
- row offset resolution,

- direct seek into `table.rows`,
- deserialization of one row.

7.2.3 Path 3: Sequential Scan

All other single-table predicates are evaluated by scanning the row file using `storage_scan_open` and `storage_scan_next`. Each row is:

- deserialized from the file stream,
- filtered for expiration if needed,
- tested against the `WHERE` clause,
- projected into the output row.

7.2.4 Path 4: Indexed Join

For `INNER JOIN` queries, the executor looks for a specific fast path:

- the join operator must be `=`,
- the join must compare columns from different tables,
- one side of the join condition must reference column index `0`.

When this holds, the executor scans one table and probes the other table through its primary index.

```
scan non-indexed side -> read join key -> primary index lookup on other
    side
```

This is significantly cheaper than a full nested-loop join when the join predicate aligns with the indexed key.

7.2.5 Path 5: Nested-Loop Join

If the indexed join fast path does not apply, the executor performs:

- an outer scan on the left table,
- an inner scan on the right table for every outer row,
- join condition evaluation for each row pair,
- optional post-join `WHERE` filtering.

7.3 Projection Handling

The executor resolves output columns before scanning data. It supports:

- `SELECT *`,
- unqualified column names when unambiguous,
- qualified names such as `TEST_USERS.NAME`.

Each output column is converted into an internal reference identifying:

- source table 1 or 2,
- column index within that table.

7.4 Ordering

`ORDER BY` is implemented by materializing result rows in memory and sorting them with a merge-sort style routine. Two design consequences follow:

- ordered queries cannot stream rows immediately,
- the ordered column must be present in the output list.

When no ordering is requested and the query is not being cached, rows can be streamed directly to the callback.

8 Storage Engine

8.1 Directory Layout

The storage layer creates and uses:

- `data/tables`
- `data/indexes`
- `data/wal`

Each table owns three logical artifacts:

- `<table>.meta`
- `<table>.rows`
- `<table>.idx`

8.2 Metadata File

The metadata file begins with a magic string and then stores:

- table name,
- column count,
- row count,
- expiration column index,
- one line per column with name and declared type.

This lets the server reconstruct the table catalog during startup without scanning row files.

8.3 Row File Format

Rows are serialized in a compact binary format:

```
[field_count:uint32]
    repeated field_count times:
        [value_length:uint32][value_bytes]
```

The system stores all row values as strings on disk. Type information is enforced at insert time rather than encoded in a typed binary row layout.

8.4 Insert Path

On insert:

- the target table is located in memory,
- the number of supplied values must match the column count,
- every supplied value is validated against the declared type,
- serialized row bytes are appended to the row file,
- the first column value and row offset are inserted into the index file,
- the metadata row count is updated and flushed.

8.5 Scan Path

Sequential scans use a buffered reader. The scan state owns:

- file pointer,
- large read buffer,
- reusable row value array,
- reusable string buffer for field contents.

This design reduces allocation churn during full-table scans.

9 Type System and Validation

The storage layer enforces a small set of types:

- `INT`
- `DECIMAL`
- `DATETIME`
- `VARCHAR(n)`

Validation rules:

- `INT`: must parse as an integer-valued number.
- `DECIMAL`: must parse numerically.
- `DATETIME`: accepts either a numeric epoch or the textual shape `YYYY-MM-DD HH:MM:SS`.
- `VARCHAR(n)`: must not exceed the declared limit.

Even though values are stored as strings, type validation prevents obviously invalid inserts.

10 Primary-Key Index Design

10.1 Index Role

Every table automatically gets one persisted B+ tree index over the first declared column. There are no secondary indexes in the current implementation.

10.2 Index File Structure

The index file stores:

- a file header with magic value, root page number, and next page number,
- fixed-size pages,
- leaf and internal nodes in B+ tree form.

The implementation uses:

- fixed key size of 64 bytes,
- up to 31 keys per page,
- a page cache inside the index subsystem for recently touched pages.

10.3 Lookup and Insert Behavior

- **Lookup:** traverse from root to leaf and return the row offset when the key exists.
- **Insert:** descend recursively, insert into a leaf, split when full, and propagate split keys upward.
- **Reset:** recreate or clear the logical contents of the index when a table is cleared.

10.4 Index Diagram

Figure 3 is intended to hold the rendered Mermaid index-structure diagram. Export your Mermaid diagram as a PNG or PDF and place it at:

- `diagrams/index_structure.png`

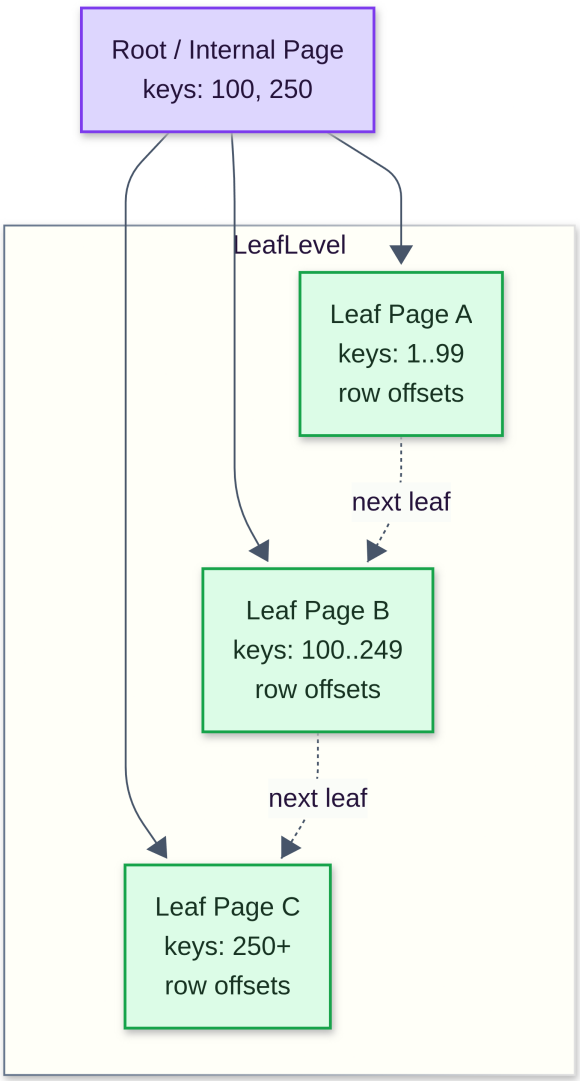

Figure 3: Conceptual primary-key B+ tree layout used by FlexQL index files.

11 Caching Strategy

11.1 Purpose

The query cache stores complete result sets for recently executed `SELECT` statements. The key is the literal SQL string.

11.2 Structure

Each cache entry contains:

- SQL text,
- output column names,
- fully copied result rows,
- LRU tick,
- occupancy flag.

The server initializes the cache with capacity 16.

11.3 Policy

- On a hit, the entry tick is refreshed.
- On insert into a full cache, the least-recently-used entry is replaced.
- On any successful mutating statement, the full cache is invalidated.
- Queries touching expiring rows are not cached.

11.4 Tradeoff

This cache is simple and effective for repeated identical reads, but it duplicates entire result sets in memory and does not support partial invalidation.

12 Expiration Semantics

If a column named `EXPIRES_AT` exists, its column index is stored in table metadata. During reads:

- the value is parsed as an epoch timestamp,
- rows with `EXPIRES_AT < now()` are skipped,
- expired rows remain on disk and in the index until the table is cleared.

This is a read-time visibility rule, not a background garbage collection mechanism.

13 Write-Ahead Logging and Recovery

13.1 WAL Format

The WAL file is stored at `data/wal/flexql.wal`. Each record is:

```
[sql_length:uint32][sql_bytes]
```

13.2 When WAL Is Used

The executor writes to the WAL before:

- `CREATE TABLE`
- `INSERT`
- `DELETE FROM table`
- `RESET table`

Pure reads do not touch the WAL.

13.3 Recovery Process

At server startup:

- storage and cache are initialized,
- the WAL file is opened,
- each stored SQL statement is replayed through the executor in recovery mode,
- the WAL is cleared after successful replay.

13.4 Recovery Diagram

Figure 4 is intended to hold the rendered Mermaid WAL recovery diagram. Export your Mermaid diagram as a PNG or PDF and place it at:

- `diagrams/wal_recovery.png`

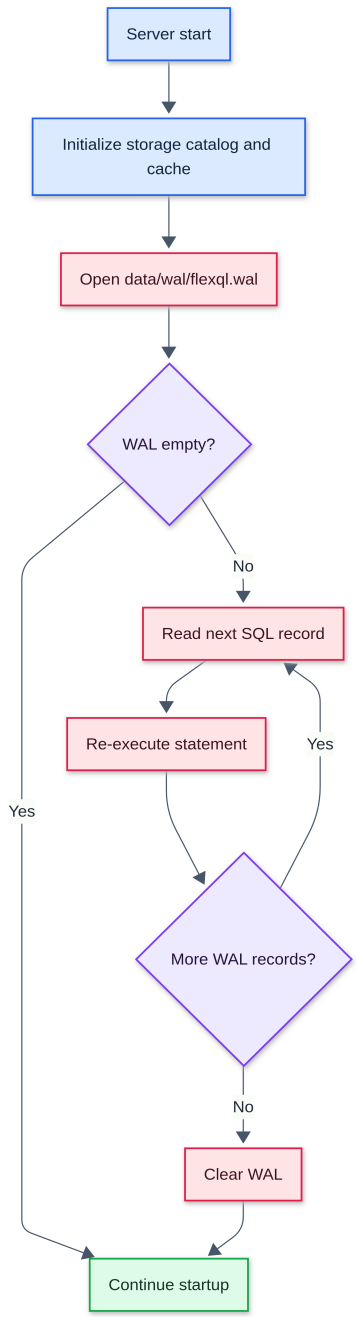

Figure 4: Startup recovery flow using the write-ahead log.

14 Concurrency Model

14.1 Server Concurrency

The server follows a thread-per-connection model:

- the accept loop listens on a single socket,
- each accepted client is handed to a detached worker thread,
- every worker receives SQL requests in a loop until disconnect.

14.2 Locking Policy

The server owns one process-wide reader/writer lock for database operations:

- **read lock** for statements whose first token is `SELECT`,
- **write lock** for all other statements.

This means:

- multiple readers can execute concurrently,
- writers are serialized,
- reads and writes do not overlap.

14.3 Cache Synchronization

The cache has its own mutex, because worker threads may read or update cache entries concurrently while still respecting the outer database lock policy.

14.4 Concurrency Diagram

Figure 5 is intended to hold the rendered Mermaid concurrency diagram. Export your Mermaid diagram as a PNG or PDF and place it at:

- `diagrams/concurrency_model.png`

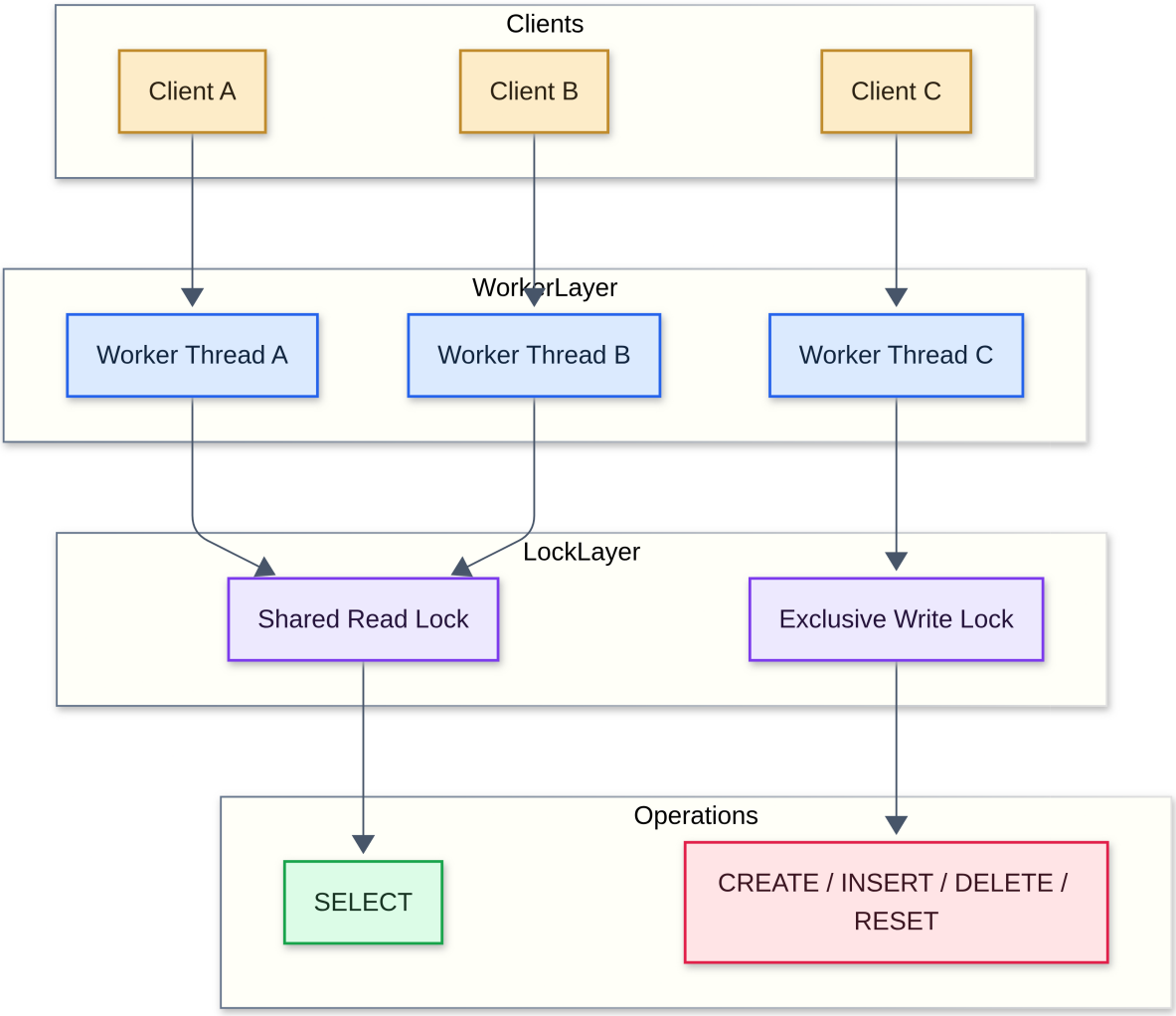

Figure 5: Thread-per-connection server model with shared read access and serialized writes.

15 Server and Client API

15.1 Public API

The public API is intentionally small:

```
int flexql_open(const char* host, int port, flexql** db);
int flexql_close(flexql* db);
int flexql_exec(flexql* db, const char* sql,
                flexql_callback callback, void* arg, char** errmsg);
void flexql_free(void* ptr);
```

15.2 Behavior

- `flexql_open` validates the endpoint and opens a TCP connection.
- `flexql_exec` sends SQL to the server and routes result rows through the callback.
- `flexql_close` closes the connection and frees client-side resources.

- `flexql_free` releases error strings returned to the caller.

The current user-facing path is remote execution through the local server. The internal handle contains fields for a local mode, but the normal public workflow in this repository is the TCP client/server path.

16 Testing and Benchmarking

16.1 Regression Test Driver

`tests/test_driver.c` validates:

- table creation,
- insertion,
- equality lookup on the primary key,
- range predicates and filtered scans,
- joins,
- ordering constraints,
- error behavior for unknown columns and invalid `ORDER BY`.

16.2 Benchmarks

The repository also includes benchmark programs that exercise:

- large inserts,
- mixed correctness validation and query timing,
- large-table join performance,
- post-insert query patterns including indexed and non-indexed predicates.

These are useful both for performance measurement and for understanding intended usage patterns.

17 Known Limitations

The following limitations are visible directly from the current implementation:

- only loopback networking is accepted,
- only one automatic primary index exists per table,
- no `UPDATE`, `DROP TABLE`, `ALTER TABLE`, aggregation, grouping, or transactions,
- table clearing is coarse-grained; there is no predicate-based delete,
- `ORDER BY` is single-column and requires the ordered column in the projection list,
- the cache is whole-query and whole-result-set only,
- expired rows are filtered at read time but not physically removed in the background,

- concurrency is controlled by one database-wide reader/writer lock rather than finer-grained table locks.

18 Summary

FlexQL is best understood as a compact educational database engine with enough real systems structure to demonstrate persistence, parsing, indexing, recovery, caching, and concurrent request handling. Its design is intentionally narrow, but the implementation is substantial enough to support meaningful testing and benchmarking. The current repository already contains the core pieces of a small disk-backed SQL engine, and the architecture is clear enough to extend with future work such as updates, secondary indexes, better query planning, or finer-grained concurrency control.